Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»



Кафедра 09.02.07_Информационные системы и программирование

ПРАКТИЧЕСКАЯ РАБОТА № 6

Дисциплина(модуль)/предмет МДК 03.02 Обеспечение качества функционирования компьютерных систем

Тема: Комплексное тестирование программных продуктов и составление отчета о тестировании (ISO/IEC 25010)

ВАРИАНТ № 10 «Общежитие и заселение» (DormitoryCheckIn)

Студент (ка)/учащийся: Леонов Тимофей Андреевич /_____ /
Ф. И. О. подпись

Группа: И14-23-9

Руководитель: _____ /_____ /
Должность, степень, Фамилия и инициалы подпись

Дата представления работы «27» октября 2026г.

Работа аттестована с оценкой _____
прописью

Санкт-Петербург 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....3
1 ОБЪЕКТ ТЕСТИРОВАНИЯ.....5
1.1 Модуль «Общежитие и заселение».....5
1.2 Структура решения.....6
2 ПЛАН ТЕСТИРОВАНИЯ.....7
2.1 Характеристики качества по ISO/IEC 25010.....7
2.2 Уровни и виды тестирования.....7
2.3 Тестовое окружение.....9
2.4 Тестовые сценарии.....10
3 РЕЗУЛЬТАТЫ ВЫПОЛНЕНИЯ ТЕСТОВ.....11
3.1 Статистика выполнения.....11
3.2 Покрытие кода.....12
4 АНАЛИЗ ДЕФЕКТОВ И РИСКОВ.....13
4.1 Дефект совместимости среды выполнения.....13
4.2 Риски.....14
5 ОТЧЕТ О ТЕСТИРОВАНИИ ПРОГРАММНОГО ПРОДУКТА.....15
ЗАКЛЮЧЕНИЕ.....17
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....18
ПРИЛОЖЕНИЕ А Исходный код модуля и тестов.....20

ВВЕДЕНИЕ

Модуль «Общежитие и заселение» предназначен для учёта проживания студентов в общежитиях учебного заведения. Он принимает через REST API заявки на заселение студентов и хранит сведения о том, кто проживает в каком корпусе и комнате. Ошибка в этом модуле приводит к потере записи о заселении или к сохранению некорректных данных, например номера несуществующего корпуса.

Актуальность темы определяется тем, что перед передачей модуля в эксплуатацию необходимо доказательно подтвердить его качество. Стандарт ГОСТ Р ИСО/МЭК 25010-2015 задаёт модель качества программного продукта, в которой функциональная пригодность и надёжность являются основными характеристиками [1], а ГОСТ Р 56920-2016 определяет понятия и процесс тестирования, итогом которого является отчёт о тестировании [2]. Автоматизированные интеграционные тесты позволяют повторять такую проверку при каждом изменении кода.

Объект исследования – модуль «Общежитие и заселение» (DormitoryCheckIn). Предмет исследования – функциональная пригодность и надёжность REST API модуля и методика их проверки автоматизированными тестами.

Новизна работы состоит в том, что каждый тест выполняется на собственной базе данных SQLite в оперативной памяти, создаваемой фабрикой тестового приложения, поэтому тесты не зависят друг от друга и от порядка запуска, а изоляция проверяется отдельным тестом.

Практическая значимость: разработанный набор тестов и отчёт позволяют принять решение о передаче модуля в эксплуатацию и использовать тесты для регрессионной проверки последующих версий.

Цель работы – выполнить комплексное тестирование модуля «Общежитие и заселение» и составить отчёт о тестировании,

подтверждающий функциональную пригодность и надёжность API по ISO/IEC 25010.

Для достижения цели поставлены задачи:

– разработать модуль API с эндпоинтами POST и GET /api/v1/dormitory/checkin на ASP.NET Core;

– составить план тестирования с выбором характеристик качества, уровней и видов тестов;

– реализовать позитивный, негативный и интеграционный тесты на xUnit с изоляцией базы данных;

– выполнить тесты, собрать статистику и покрытие кода;

– проанализировать выявленные дефекты и риски, сформулировать итоговый вердикт.

Методы работы: анализ требований, моделирование тестового окружения в draw.io, интеграционное тестирование через WebApplicationFactory, тестирование классами эквивалентности и граничными значениями, измерение покрытия кода, анализ первопричин дефектов.

1 ОБЪЕКТ ТЕСТИРОВАНИЯ

1.1 Модуль «Общежитие и заселение»

Объектом тестирования является модуль Campus.Dormitory.Api – веб-API на ASP.NET Core (.NET 10) с хранением данных в SQLite через Entity Framework Core. Модуль работает с сущностью DormitoryCheckIn (листинг 1).

Листинг 1 – Сущность DormitoryCheckIn

```
public class DormitoryCheckIn
{
   public int Id { get; set; }

   [Required]
   public string? ResidentPassport { get; set; }

   [Range(1, 9)]
   public int BuildingNumber { get; set; }

   public int RoomNumber { get; set; }
}
```

Требования к данным заданы атрибутами валидации: паспорт проживающего обязателен, номер корпуса должен лежать в диапазоне от 1 до 9. Контроллер помечен атрибутом [ApiController], поэтому при нарушении этих требований ASP.NET Core автоматически возвращает ответ 400 Bad Request с объектом ValidationProblemDetails и не вызывает метод контроллера [5]. Эндпоинты модуля приведены в таблице 1.

Таблица 1 – Эндпоинты модуля

Метод и адрес	Назначение	Ожидаемые ответы
POST /api/v1/dormitory/checkin	регистрация заселения, тело запроса – DormitoryCheckIn в JSON	200 OK с созданной записью; 400 Bad Request при ошибке валидации
GET /api/v1/dormitory/checkin	список всех заселений, упорядоченный по Id	200 OK с массивом записей

Источник: составлено автором

1.2 Структура решения

Решение Microsoft Visual Studio Campus.Dormitory.sln размещено в папке pr6 репозитория https://github.com/kotokotru/mdk-03-02. Состав решения приведён в таблице 2, исходный код – в приложении А.

Таблица 2 – Состав решения Campus.Dormitory.sln

Проект и файл	Назначение
src/Campus.Dormitory.Api/Models/ DormitoryCheckIn.cs	сущность заселения с атрибутами валидации
src/Campus.Dormitory.Api/Data/ DormitoryDbContext.cs	контекст EF Core, набор CheckIns
src/Campus.Dormitory.Api/Controllers/ DormitoryController.cs	эндпоинты POST и GET
src/Campus.Dormitory.Api/Program.cs	настройка сервисов, SQLite, создание базы
tests/Campus.Dormitory.Tests/ CustomWebApplicationFactory.cs	фабрика тестового приложения с SQLite In-Memory
tests/Campus.Dormitory.Tests/ DormitoryCheckInTests.cs	три теста по заданию

Источник: составлено автором

2 ПЛАН ТЕСТИРОВАНИЯ

2.1 Характеристики качества по ISO/IEC 25010

Модель качества продукта ISO/IEC 25010 включает восемь характеристик [1]. Для модуля, который принимает и хранит данные о заселении, выбраны характеристики и подхарактеристики, приведённые в таблице 3.

Таблица 3 – Проверяемые характеристики качества

Характеристика (подхарактеристика)	Что означает для модуля	Тест
Функциональная пригодность (функциональная полнота, корректность)	корректная заявка сохраняется и возвращается списком без искажений	Тест 1
Надёжность (отказоустойчивость)	некорректный ввод не приводит к сбою и не попадает в базу, клиент получает понятный ответ 400	Тест 2
Надёжность (зрелость), сопровождаемость (тестируемость)	результат теста не зависит от других тестов и порядка их запуска	Тест 3

Источник: составлено автором

2.2 Уровни и виды тестирования

План тестирования охватывает три уровня проверки, перечисленные в задании (таблица 4).

Таблица 4 – Уровни и виды тестирования

Уровень (вид)	Содержание	Реализация
Unit (модульный)	правила сущности: обязательность паспорта и диапазон корпуса 1–9	атрибуты [Required] и [Range], проверяются тестом 2 через конвейер ASP.NET Core
Validation (проверка входных данных)	отклонение null и значений вне диапазона кодом 400	тест 2, классы эквивалентности и граничное значение 10
Integration (интеграционный)	цепочка HTTP-запрос – контроллер – EF Core – SQLite	тесты 1 и 3 через WebApplicationFactory и TestServer

Источник: составлено автором

Для теста 2 выбраны значения из недопустимых классов эквивалентности [2]: отсутствующий паспорт (null) и номер корпуса 10, который является ближайшим недопустимым значением за верхней границей диапазона 1–9. Оба нарушения передаются в одном запросе, и тест проверяет, что сервер сообщает о каждом из них отдельно.

Порядок проведения тестирования показан на рисунке 1. Если после запуска есть проваленные тесты, выполняется анализ дефектов с разделением на первичную ошибку и вторичные симптомы, устраняется причина и тесты запускаются повторно.

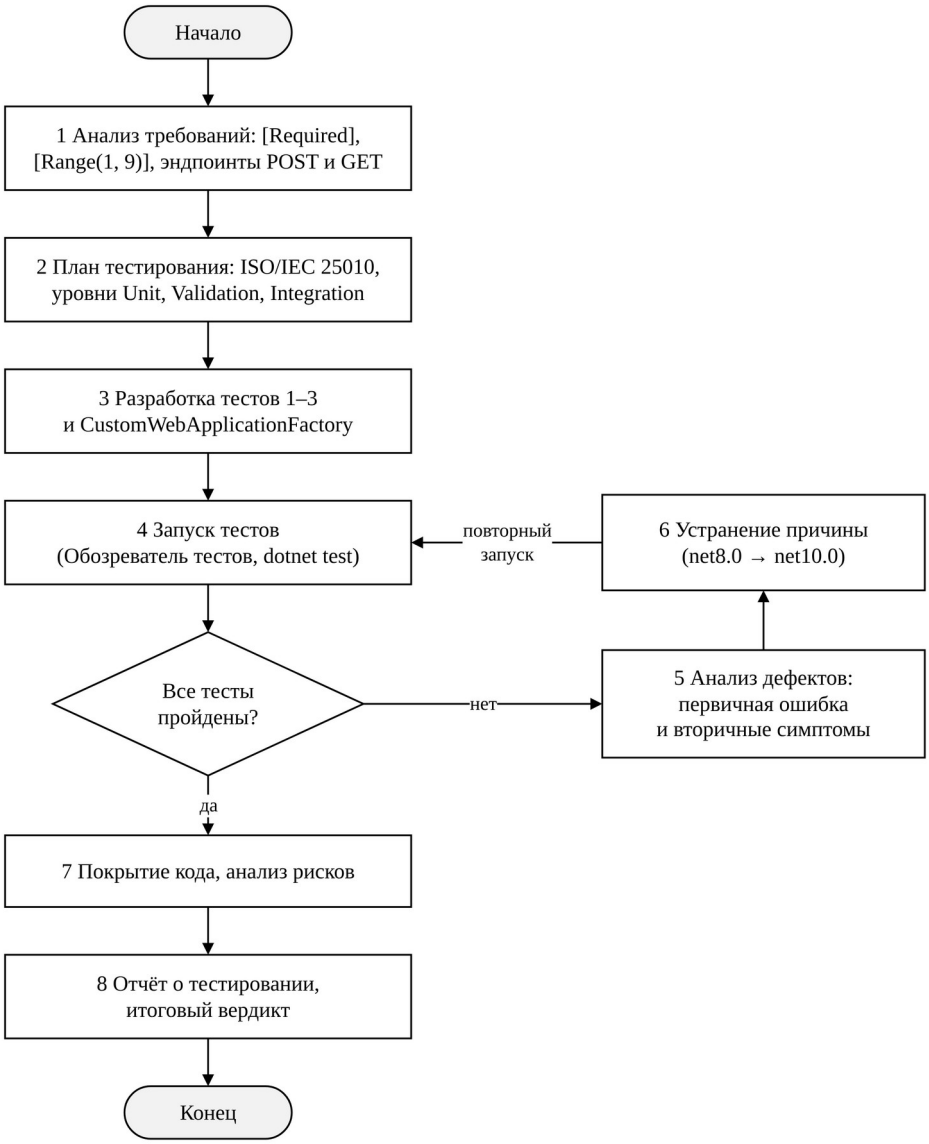


Рисунок 1 – Блок-схема процесса тестирования

2.3 Тестовое окружение

Тесты выполняются без запуска веб-сервера и без сети. Класс CustomWebApplicationFactory наследует WebApplicationFactory<Program> и поднимает приложение в памяти на TestServer [3]. В методе ConfigureWebHost фабрика удаляет регистрацию рабочей базы dormitory.db и подключает DbContext к открытому соединению SQLite с адресом DataSource=:memory: [4]. База в памяти существует, пока открыто соединение, поэтому у каждой фабрики своя чистая база, которая уничтожается при Dispose.

xUnit создаёт новый экземпляр тестового класса для каждого теста [6], а конструктор класса создаёт новую фабрику. Схема тестового окружения приведена на рисунке 2.

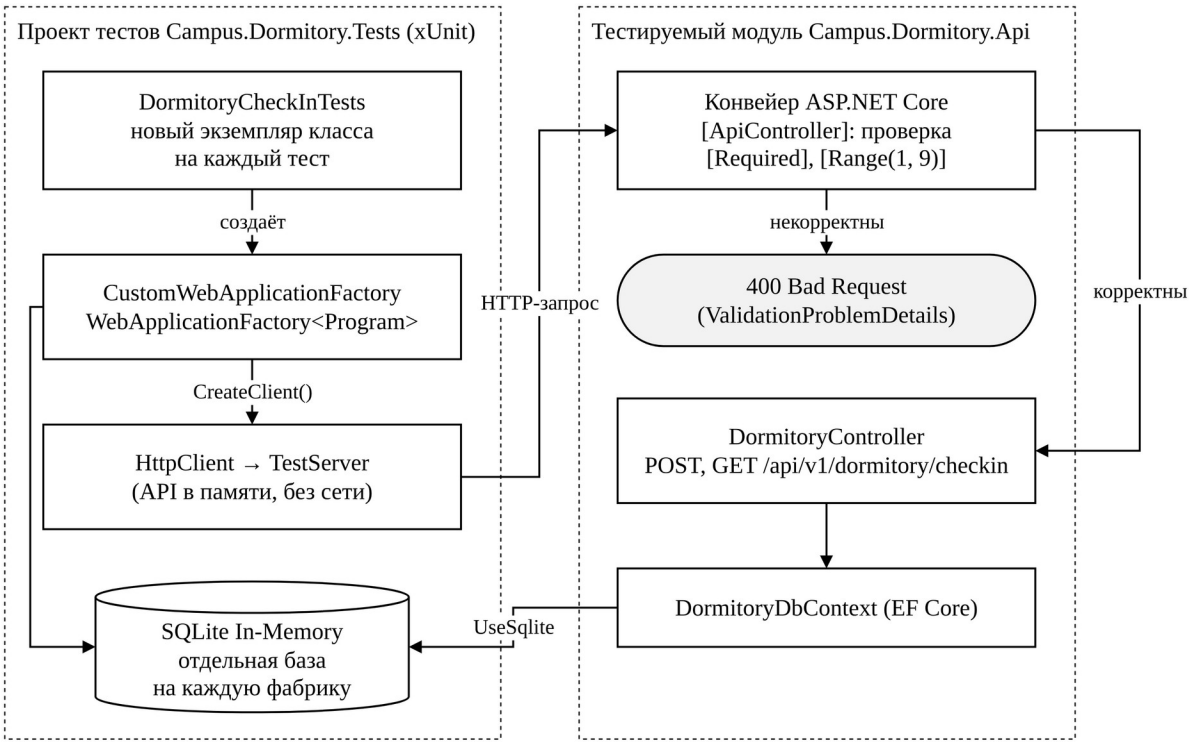


Рисунок 2 – Схема тестового окружения

Таблица 5 – Программные средства

Средство	Версия	Назначение
.NET SDK, ASP.NET Core	10.0	платформа модуля и тестов
Microsoft. EntityFrameworkCore.Sqlite	10.0.0	хранение данных, SQLite In-Memory в тестах
Microsoft.AspNetCore.Mvc. Testing	10.0.0	WebApplicationFactory, TestServer
xunit, xunit.runner.visualstudio	2.9.2, 2.8.2	фреймворк тестирования, запуск в Visual Studio
coverlet.collector	6.0.2	измерение покрытия кода
Microsoft Visual Studio Community	2026	среда разработки, обозреватель тестов

Источник: составлено автором

2.4 Тестовые сценарии

Тестовые сценарии с входными данными и ожидаемыми результатами приведены в таблице 6.

Таблица 6 – Тестовые сценарии

Тест и метод	Входные данные	Ожидаемый результат
Тест 1. PostCheckIn_ValidData_ Returns200AndSavesRecord	ResidentPassport = «4015 123456», BuildingNumber = 3, RoomNumber = 412	POST – 200 OK, Id > 0; GET – 200 OK, одна запись с теми же данными
Тест 2. PostCheckIn_ NullPassportAndInvalidBuilding_ Returns400BadRequest	ResidentPassport = null, BuildingNumber = 10, RoomNumber = 101	400 Bad Request, ошибки по полям ResidentPassport и BuildingNumber; GET – пустой список
Тест 3. Database_ IsEmptyAndIsolated_ ForEachTestAndFactory	запись «4020 654321», корпус 1, комната 7 во второй фабрике	база теста пуста до и после; во второй фабрике ровно одна запись

Источник: составлено автором

3 РЕЗУЛЬТАТЫ ВЫПОЛНЕНИЯ ТЕСТОВ

3.1 Статистика выполнения

Тесты запущены в обозревателе тестов Microsoft Visual Studio и командой dotnet test. Все три теста пройдены (рисунок 3), в обозревателе тестов предупреждений и ошибок нет. Сводка по группе тестов приведена на рисунке 4.

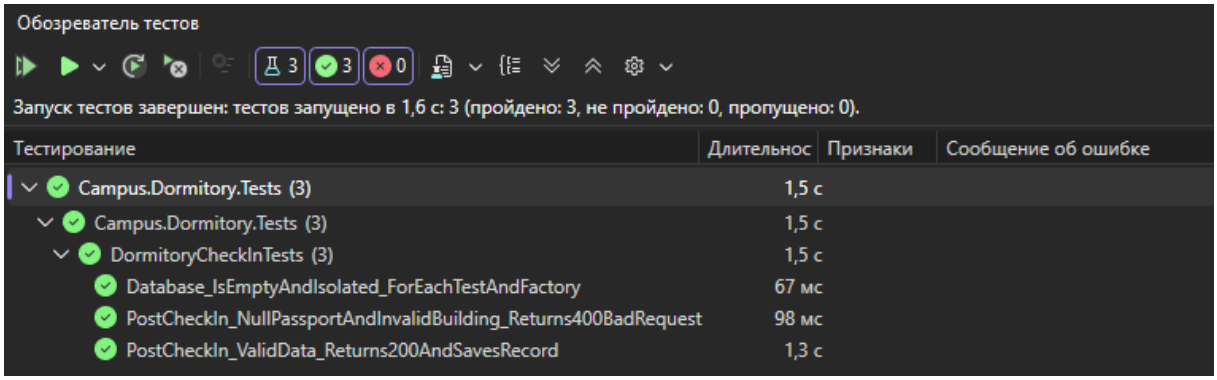


Рисунок 3 – Обозреватель тестов Visual Studio: 3 теста пройдено

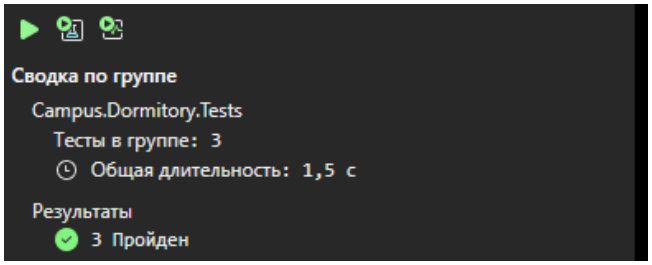


Рисунок 4 – Сводка по группе тестов Campus.Dormitory.Tests

Таблица 7 – Статистика выполнения тестов

Тест	Длительность (Visual Studio)	Длительность (dotnet test)	Результат
Тест 1 (позитивный)	1,3 с	1 с	Passed
Тест 2 (негативный)	98 мс	77 мс	Passed
Тест 3 (изоляция)	67 мс	70 мс	Passed
Итого: 3 теста	1,5 с	4,2 с с учётом запуска	3 Passed, 0 Failed

Источник: составлено автором

Первый тест выполняется дольше остальных, так как в нём впервые запускается приложение и инициализируются EF Core и SQLite. Фрагмент вывода dotnet test приведён в листинге 2.

Листинг 2 – Вывод команды dotnet test

```
[xUnit.net 00:00:02.28]   Finished:    Campus.Dormitory.Tests
  Пройден ...PostCheckIn_ValidData_Returns200AndSavesRecord [1 s]
  Пройден ...PostCheckIn_NullPassportAndInvalidBuilding_Returns400BadRequest [77 ms]
  Пройден ...Database_IsEmptyAndIsolated_ForEachTestAndFactory [70 ms]
Тестовый запуск выполнен.
Всего тестов: 3
     Пройдено: 3
 Общее время: 4,1994 Секунды
```

3.2 Покрытие кода

Покрытие измерено сборщиком coverlet командой dotnet test --collect:"XPlat Code Coverage" [7]. Результаты приведены в таблице 8.

Таблица 8 – Покрытие кода модуля Campus.Dormitory.Api

Показатель	Покрыто	Всего	Процент
Строки	23	23	100 %
Ветвления	1	2	50 %

Источник: составлено автором

Все строки модуля выполняются тестами. Непокрытая ветвь – оператор ?? в Program.cs, который подставляет строку подключения по умолчанию, если она не задана в конфигурации. Эта ветвь относится к настройке запуска, а не к логике заселения, и на результат тестирования не влияет.

4 АНАЛИЗ ДЕФЕКТОВ И РИСКОВ

4.1 Дефект совместимости среды выполнения

При первом запуске все три теста провалились. Для анализа дефекты разделены на первичную ошибку и вторичные симптомы (таблица 9).

Таблица 9 – Анализ дефекта первого запуска

Категория	Описание
Первичная ошибка (причина)	проект был собран под .NET 8, а на компьютере установлен только runtime .NET 10; приложение запустилось на нём с повышением версии (roll-forward), и SystemTextJsonOutputFormatter из ASP.NET Core 8 выбрасывал InvalidOperationException: «The PipeWriter 'ResponseBodyPipeWriter' does not implement PipeWriter.UnflushedBytes» при записи любого JSON-ответа
Вторичный симптом – тест 1	команда INSERT выполнена и запись сохранена, но вместо ответа 200 тест получил исключение сервера, которое TestServer пробрасывает в вызывающий код
Вторичный симптом – тест 2	валидация сработала верно, но ответ 400 с ValidationProblemDetails не удалось сериализовать
Вторичный симптом – тест 3	падение на первом запросе GET при чтении пустого списка
Исправление	решение переведено на целевую платформу net10.0, пакеты EF Core и Mvc.Testing – на версию 10.0.0
Повторная проверка	3 из 3 тестов пройдены

Источник: составлено автором

Анализ показывает, что три падения вызваны одной причиной в среде выполнения, а не ошибками бизнес-логики: сообщение об ошибке и стек вызовов во всех трёх тестах совпадают, а логика валидации и изоляции до сбоя отработала правильно. Если бы дефекты рассматривались по отдельности, можно было бы ошибочно исправлять контроллер или тесты.

4.2 Риски

Главный риск выявлен при сборке: NuGet выдаёт предупреждение NU1903 об уязвимости транзитивного пакета SQLitePCLRaw.lib.e_sqlite3 (рисунок 5).

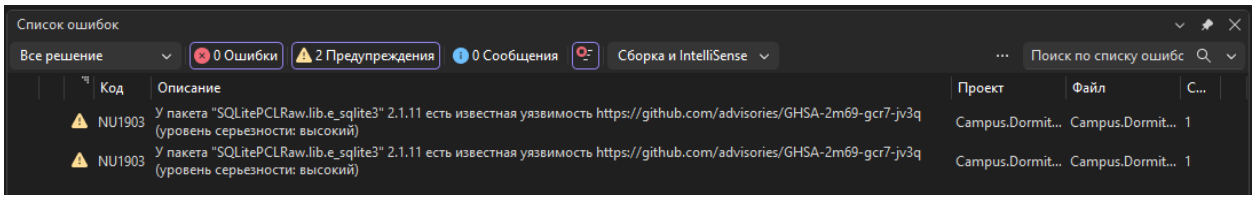


Рисунок 5 – Предупреждения NU1903 в списке ошибок Visual Studio

Выявленные риски и меры по их снижению приведены в таблице 10.

Таблица 10 – Риски

Риск	Оценка	Мера
уязвимость транзитивного пакета SQLitePCLRaw.lib.e_sqlite3 2.1.11 (предупреждение NuGet NU1903, GHSA-2m69-gcr7-jv3q) [8]	высокий уровень по данным NuGet	обновить пакет до исправленной версии, включить проверку уязвимостей в CI
расхождение версии целевой платформы и установленного runtime на сервере	средний	фиксировать версию SDK в global.json, развёртывать с runtime той же версии
SQLite в тестах отличается от промышленной СУБД, если она будет заменена	низкий	при смене СУБД добавить интеграционные тесты на ней
номер комнаты не ограничен валидацией (возможно 0 или отрицательное значение)	средний	согласовать с заказчиком диапазон RoomNumber и добавить атрибут и тест

Источник: составлено автором

5 ОТЧЕТ О ТЕСТИРОВАНИИ ПРОГРАММНОГО ПРОДУКТА

Отчёт составлен по шаблону задания.

1. ФИО студента: Леонов Тимофей Андреевич, группа И14-23-9.

2. Вариант и модуль: вариант 10, модуль «Общежитие и заселение» (DormitoryCheckIn).

3. Цель тестирования: проверка функциональной пригодности и надежности API (ISO/IEC 25010).

4. Итоговая статистика (Test Summary) приведена в таблице 11.

Таблица 11 – Итоговая статистика (Test Summary)

Показатель	Значение
Всего тестов	3
Успешно (Passed)	3
Провалено (Failed)	0
Статус готовности модуля	ПЕРЕДАН

Источник: составлено автором

5. Описание выполненных тестов приведено в таблице 12.

Таблица 12 – Описание выполненных тестов

Тест	Метод	Проверка	Результат
Тест 1 (позитивный, Functional Suitability)	PostCheckIn_ValidData_ Returns200AndSavesRecord	корректная заявка возвращает 200 OK и сохраняется, GET возвращает её без искажений	Passed
Тест 2 (негативный, Validation)	PostCheckIn_ NullPassportAndInvalidBuildi ng_Returns400BadRequest	null в паспорте и корпус 10 возвращают 400 Bad Request с ошибками по обоим полям, запись не сохраняется	Passed
Тест 3 (интеграционный, Isolation)	Database_ IsEmptyAndIsolated_ ForEachTestAndFactory	каждая фабрика использует свою базу SQLite In-Memory, данные одного теста не видны другому	Passed

Источник: составлено автором

6. Скриншоты Test Explorer приведены на рисунках 3 и 4 (подраздел 3.1).

7. Заключение (вывод): все запланированные тесты пройдены. Первичная ошибка первого запуска – несовместимость версий .NET 8 и .NET 10 – устранена переводом решения на .NET 10, вторичные симптомы исчезли. Модуль соответствует требованиям функциональной пригодности и

надёжности и готов к развёртыванию с учётом рекомендации по обновлению пакета SQLitePCLRaw.

ЗАКЛЮЧЕНИЕ

В ходе практической работы выполнено комплексное тестирование модуля «Общежитие и заселение» и составлен отчёт о тестировании.

Разработан модуль API на ASP.NET Core с эндпоинтами POST и GET /api/v1/dormitory/checkin. Составлен план тестирования: выбраны характеристики качества ISO/IEC 25010 (функциональная пригодность и надёжность), определены уровни Unit, Validation и Integration, спроектировано тестовое окружение на WebApplicationFactory и SQLite In-Memory, схема которого выполнена в draw.io.

Реализованы три теста на xUnit: позитивный, негативный и интеграционный тест изоляции. Все тесты пройдены, покрытие строк модуля – 100 %. Дефект первого запуска проанализирован с разделением на первичную ошибку и вторичные симптомы и устранён. Выявлены риски, главный из которых – уязвимость транзитивного пакета SQLite, для каждого предложена мера.

Итоговый вердикт: модуль «Общежитие и заселение» готов к развёртыванию, статус – ПЕРЕДАН.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ Р ИСО/МЭК 25010-2015. Информационные технологии. Системная и программная инженерия. Требования и оценка качества систем и программного обеспечения (SQuaRE). Модели качества систем и программных продуктов : национальный стандарт Российской Федерации : дата введения 2016-06-01. – URL: https://files.stroyinf.ru/Index/60/60038.htm (дата обращения: 27.10.2026). – Текст : электронный.

2. ГОСТ Р 56920-2016/ISO/IEC/IEEE 29119-1:2013. Системная и программная инженерия. Тестирование программного обеспечения. Часть 1. Понятия и определения : национальный стандарт Российской Федерации : дата введения 2017-06-01. – URL: https://docs.cntd.ru/document/1200134996 (дата обращения: 27.10.2026). – Текст : электронный.

3. Интеграционные тесты в ASP.NET Core // Microsoft Learn : сайт. – URL: https://learn.microsoft.com/ru-ru/aspnet/core/test/integration-tests (дата обращения: 27.10.2026). – Текст : электронный.

4. Выбор стратегии тестирования. Тестирование с использованием SQLite в памяти // Microsoft Learn : сайт. – URL: https://learn.microsoft.com/ru-ru/ef/core/testing/choosing-a-testing-strategy (дата обращения: 27.10.2026). – Текст : электронный.

5. Проверка модели в ASP.NET Core MVC и Razor Pages // Microsoft Learn : сайт. – URL: https://learn.microsoft.com/ru-ru/aspnet/core/mvc/models/validation (дата обращения: 27.10.2026). – Текст : электронный.

6. Shared Context between Tests // xUnit.net : сайт. – URL: https://xunit.net/docs/shared-context (дата обращения: 27.10.2026). – Текст : электронный.

7. Использование покрытия кода для модульного тестирования // Microsoft Learn : сайт. – URL:

https://learn.microsoft.com/ru-ru/dotnet/core/testing/unit-testing-code-coverage (дата обращения: 27.10.2026). – Текст : электронный.

8. NuGet Warning NU1903 // Microsoft Learn : сайт. – URL: https://learn.microsoft.com/ru-ru/nuget/reference/errors-and-warnings/nu1901-nu1904 (дата обращения: 27.10.2026). – Текст : электронный.

ПРИЛОЖЕНИЕ А

Исходный код модуля и тестов

В приложении приведён исходный код модуля Campus.Dormitory.Api и проекта тестов Campus.Dormitory.Tests (C#, .NET 10, решение Microsoft Visual Studio Campus.Dormitory.sln).

Листинг А.1 – Файл src/Campus.Dormitory.Api/Program.cs

```
using Campus.Dormitory.Api.Data;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddDbContext<DormitoryDbContext>(options =>
    options.UseSqlite(builder.Configuration.GetConnectionString("Dormitory") ?? "Data Source=dormitory.db"));

var app = builder.Build();

using (var scope = app.Services.CreateScope())
    scope.ServiceProvider.GetRequiredService<DormitoryDbContext>().Database.EnsureCreated();

app.MapControllers();
app.Run();

// нужен для WebApplicationFactory<Program> в проекте тестов
public partial class Program;
```

Листинг А.2 – Файл src/Campus.Dormitory.Api/Models/DormitoryCheckIn.cs

```
using System.ComponentModel.DataAnnotations;

namespace Campus.Dormitory.Api.Models;

/// <summary>Заселение студента в общежитие (вариант 10).</summary>
public class DormitoryCheckIn
{
    public int Id { get; set; }

    [Required] public string? ResidentPassport { get; set; }

    [Range(1, 9)] public int BuildingNumber { get; set; }

    public int RoomNumber { get; set; }
}
```

Листинг А.3 – Файл src/Campus.Dormitory.Api/Data/DormitoryDbContext.cs

```
using Campus.Dormitory.Api.Models;
using Microsoft.EntityFrameworkCore;

namespace Campus.Dormitory.Api.Data;

public class DormitoryDbContext(DbContextOptions<DormitoryDbContext> options) : DbContext(options)
{
    public DbSet<DormitoryCheckIn> CheckIns => Set<DormitoryCheckIn>();
}
```

Листинг А.4 – Файл src/Campus.Dormitory.Api/Controllers/DormitoryController.cs

```
using Campus.Dormitory.Api.Data;
using Campus.Dormitory.Api.Models;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace Campus.Dormitory.Api.Controllers;

/// <summary>
/// Модуль «Общежитие и заселение». Атрибут [ApiController] включает автоматическую проверку модели:
/// при нарушении [Required] или [Range] клиент получает 400 Bad Request ещё до вызова метода.
/// </summary>
[ApiController]
[Route("api/v1/dormitory/checkin")]
public class DormitoryController(DormitoryDbContext db) : ControllerBase
{
    // POST /api/v1/dormitory/checkin — зарегистрировать заселение
    [HttpPost]
    public async Task<ActionResult<DormitoryCheckIn>> CheckIn(DormitoryCheckIn checkIn)
    {
        checkIn.Id = 0;                    // идентификатор назначает база данных
        db.CheckIns.Add(checkIn);
        await db.SaveChangesAsync();
        return Ok(checkIn);
    }

    // GET /api/v1/dormitory/checkin — список заселений
    [HttpGet]
    public async Task<ActionResult<List<DormitoryCheckIn>>> GetAll() =>
        Ok(await db.CheckIns.AsNoTracking().OrderBy(c => c.Id).ToListAsync());
}
```

Листинг А.5 – Файл tests/Campus.Dormitory.Tests/CustomWebApplicationFactory.cs

```
using Campus.Dormitory.Api.Data;
using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.Mvc.Testing;
using Microsoft.Data.Sqlite;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.DependencyInjection.Extensions;

namespace Campus.Dormitory.Tests;

/// <summary>
/// Поднимает API в памяти и подменяет файловую базу на SQLite In-Memory.
/// Каждый экземпляр фабрики получает своё открытое соединение, то есть свою отдельную пустую базу:
/// база живёт, пока открыто соединение, и исчезает при Dispose.
/// </summary>
public class CustomWebApplicationFactory : WebApplicationFactory<Program>
{
    private readonly SqliteConnection _connection = new("DataSource=:memory:");

    public CustomWebApplicationFactory() => _connection.Open();

    protected override void ConfigureWebHost(IWebHostBuilder builder)
    {
        builder.UseEnvironment("Testing");
        builder.ConfigureServices(services =>
        {
            // убираем регистрацию рабочей базы dormitory.db
            services.RemoveAll<DbContextOptions<DormitoryDbContext>>();
            services.AddDbContext<DormitoryDbContext>(options => options.UseSqlite(_connection));
        });
    }

    protected override void Dispose(bool disposing)
    {
        base.Dispose(disposing);
        if (disposing) _connection.Dispose();
    }
}
```

Листинг А.6 – Файл tests/Campus.Dormitory.Tests/DormitoryCheckInTests.cs

```
using System.Net;
using System.Net.Http.Json;
using Campus.Dormitory.Api.Models;
using Microsoft.AspNetCore.Mvc;

namespace Campus.Dormitory.Tests;

/// <summary>
/// Тесты модуля «Общежитие и заселение» (вариант 10).
/// xUnit создаёт новый экземпляр класса на каждый тест, поэтому у каждого теста своя фабрика и своя база.
/// </summary>
public class DormitoryCheckInTests : IDisposable
{
    private const string Url = "/api/v1/dormitory/checkin";
    private readonly CustomWebApplicationFactory _factory = new();
    private readonly HttpClient _client;

    public DormitoryCheckInTests() => _client = _factory.CreateClient();
```

```
    public void Dispose()
    {
        _client.Dispose();
        _factory.Dispose();
    }

    // Тест 1. Позитивный (Functional Suitability)
    [Fact]
    public async Task PostCheckIn_ValidData_Returns200AndSavesRecord()
    {
        var request = new DormitoryCheckIn { ResidentPassport = "4015 123456", BuildingNumber = 3,
RoomNumber = 412 };

        var post = await _client.PostAsJsonAsync(Url, request);

        Assert.Equal(HttpStatusCode.OK, post.StatusCode);
        var created = await post.Content.ReadFromJsonAsync<DormitoryCheckIn>();
        Assert.NotNull(created);
        Assert.True(created.Id > 0);
        Assert.Equal("4015 123456", created.ResidentPassport);

        var get = await _client.GetAsync(Url);
        Assert.Equal(HttpStatusCode.OK, get.StatusCode);
        var all = await get.Content.ReadFromJsonAsync<List<DormitoryCheckIn>>();
        var saved = Assert.Single(all!);
        Assert.Equal(created.Id, saved.Id);
        Assert.Equal(3, saved.BuildingNumber);
        Assert.Equal(412, saved.RoomNumber);
    }

    // Тест 2. Негативный (Validation): паспорт не передан, корпус вне диапазона 1–9
    [Fact]
    public async Task PostCheckIn_NullPassportAndInvalidBuilding_Returns400BadRequest()
    {
        var request = new DormitoryCheckIn { ResidentPassport = null, BuildingNumber = 10, RoomNumber = 101 };

        var post = await _client.PostAsJsonAsync(Url, request);

        Assert.Equal(HttpStatusCode.BadRequest, post.StatusCode);
        var problem = await post.Content.ReadFromJsonAsync<ValidationProblemDetails>();
        Assert.NotNull(problem);
        Assert.Contains(problem.Errors.Keys, k => k.Equals(nameof(DormitoryCheckIn.ResidentPassport),
StringComparison.OrdinalIgnoreCase));
        Assert.Contains(problem.Errors.Keys, k => k.Equals(nameof(DormitoryCheckIn.BuildingNumber),
StringComparison.OrdinalIgnoreCase));

        // некорректная запись не должна попасть в базу
        var all = await _client.GetFromJsonAsync<List<DormitoryCheckIn>>(Url);
        Assert.Empty(all!);
    }

    // Тест 3. Интеграционный (Isolation): у каждого теста и каждой фабрики своя чистая база
    [Fact]
    public async Task Database_IsEmptyAndIsolated_ForEachTestAndFactory()
    {
        // база этого теста пуста, хотя другие тесты класса уже добавляли записи
        var before = await _client.GetFromJsonAsync<List<DormitoryCheckIn>>(Url);
        Assert.Empty(before!);

        // запись во второй, независимой фабрике
        using (var otherFactory = new CustomWebApplicationFactory())
        using (var otherClient = otherFactory.CreateClient())
```

```
        {
            var post = await otherClient.PostAsJsonAsync(Url,
                new DormitoryCheckIn { ResidentPassport = "4020 654321", BuildingNumber = 1, RoomNumber = 7 });
            Assert.Equal(HttpStatusCode.OK, post.StatusCode);
            Assert.Single((await otherClient.GetFromJsonAsync<List<DormitoryCheckIn>>(Url))!);
        }

        // в базе текущего теста по-прежнему ничего нет
        var after = await _client.GetFromJsonAsync<List<DormitoryCheckIn>>(Url);
        Assert.Empty(after!);
    }
}
```